Consequentialism: Counterfactual Sampling to Speed Learning

1st Adrian Manchado
Diercks School of Advanced Computing
Milwaukee School of Engineering
Milwaukee, WI USA
BLARBLAR@msoe.edu

2nd Jeremy Kedziora
Diercks School of Advanced Computing
Milwaukee School of Engineering
Milwaukee, WI USA
kedziora@msoe.edu

***Abstract*—Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar.**

***Index Terms*—Blar blar blar blar**

I. Introduction

In this paper, we seek to identify consequential moments within episodes, measure their magnitude, and leverage this information to improve the sample efficiency of reinforcement learning (RL) algorithms.

II. Related Work

Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar.

III. Background

In reinforcement learning, control problems are modeled as a Markov Decision Process (MDP) [1] which is defined by:

- A set of states S that describe the current environmental conditions facing the agent;
- A set of actions $A(s)$ that an agent can take;
- Probabilities $p(s' \mid a, s)$ for transitioning from state s to state s' given action a;
- A function $r : S \times A \times S \to \mathbb{R}$ so that $r(s', a, s)$ supplies the immediate reward associated with this transition.

In environments that take place across a finite number of discrete periods T, the sequence of periods the agent participates in is referred to as an episode. The goal of the agent is to learn a policy $\pi(a|s)$, which describes the probability that a trained agent should take action a in state s, to maximize the sequence of rewards across across an episode: $\sum_{t=0}^{T} \gamma^t r(s_{t+1}, a_t, s_t)$. Here a_t and s_t are the action and state at time t and $\gamma \in [0, 1]$ is the discount factor on future rewards. Throughout we will slightly abuse notation to write $r(s_{t+1}, a_t, s_t)$ as r_{t+1}.

A. Trajectories and Returns

The choices of an agent via its policy $\pi(\cdot)$ throughout the course of an episode lead to a realized time series of information commonly referred to as a trajectory:

$$\tau_\pi = s_0, a_0, r_1, s_1, a_1, \ldots, r_{T-1}, s_{T-1}, a_{T-1}, r_T, s_T.$$

In general, it may be useful to refer to partial trajectories below. We will denote a slice of a trajectory generated by policy $\pi(\cdot)$, beginning at time t, and ending at time t' as $\tau_\pi^{(t:t')}$. Thus $\tau_\pi^{(0:t)}$ will refer to the trajectory data up until time t and $\tau_\pi^{(t:T)}$ will refer to the trajectory data from t to the end of the episode. The return at time t associated with a slice from a trajectory of data is the sum of discounted rewards within the trajectory:

$$G(\tau_\pi^{(t:t')}) = \sum_{j=t+1}^{t'} \gamma^{j-t-1} r_j.$$

Given this, for any specific time point t, the remaining return for an episode generated by $\pi(\cdot)$ can be written as $G(\tau_\pi^{(t:T)})$.

B. Measuring Important Moments

One way to conceptualize the measurement of key moments is to take a counterfactual approach and ask "what would have happened if" questions of the policy and the data generated by it. Given a trajectory τ_π at any moment t the agent took an historic action a_t according to the distribution associated with $\pi(s_t)$ leading to the realized trajectory with probability:

$$p(s_T|s_{T-1}, a_{T-1}) \prod_{j=t}^{T-1} p(s_{j+1}|s_j, a_j)\pi(a_{j+1}|s_{j+1})$$

which depends on both the policy and unobserved dynamics of the MDP. Thus, at time t, from the perspective of the agent $\tau_\pi^{(t:)} \sim d_{\tau_\pi^{(t:)}}(s_t, a_t)$, where $d_{\tau_\pi^{(t:)}}(\cdot)$ is a distribution with dependence on the action and state. Since the remaining return is entirely determined by the remaining trajectory $G(\tau_\pi^{(t:)})$ also follows $d_{\tau_\pi^{(t:)}}(\cdot)$. Formally, let:

$$\mathcal{R}(g) = \{\tau^{t:} | G(\tau^{t:}) = g\}$$

be the set of all trajectories beginning at time t whose return will equal g. Then:

$$d_G^{(\pi)}(g|s_t, a) \equiv \int_{\mathcal{R}(g)} d_{\tau_\pi^{(t:)}}(\tau|s_t, a) d\tau. \tag{1}$$

That is, the distribution over future returns $G(\tau_\pi^{(t:)})$ is derived from "adding up" the trajectories that lead to a given return.

Consider an intervention – an enforced change at time t from the historic action a_t to a feasible, alternative action $a \in A(s_t)$. Such an intervention leads to a natural comparison of two distributions, the one for the returns derived from historic action $d_G^{(\pi)}(g|s_t, a_t)$ and the one for the unrealized trajectories that would have followed the alternative action a, $d_G^{(\pi)}(g|s_t, a)$; here we consider the use of a generic metric, $m(\cdot)$, that quantifies the difference amongst a set of distributions $d_G^{(\pi)}(g|s_t, a_1), \ldots, d_G^{(\pi)}(g|s_t, a_{|A|})$.[1]

Algorithm 1 Counterfactual Consequence Estimation

Input: policy π, state-action pair (s_t, a_t), and $n \in \mathbb{N}_+$
for $a \in A(s_t)$ **do**
  Use π to sample n trajectories $\tau_{\pi,a}^{j,t:} \sim d_{\tau_\pi^{(t:)}}(s_t, a)$
  For each sampled trajectory j compute $G(\tau_{\pi,a}^{j,t:})$
  Estimate $d_G^{(\pi)}(s_t, a)$ from $G(\tau_{\pi,a}^{1,t:}), \ldots, G(\tau_{\pi,a}^{n,t:})$
end for
Compute $m(\{d_G^{(\pi)}(s_t, a)\}_{a \in A})$

IV. Variance Reduction in Monte Carlo

Blar blar.

V. Consequence Prioritization

One straightforward way to apply the sampling-based method of algorithm 1 is to use it as a way to adjust how the agent encounters data to learn from. For example, [3] extended the DQN algorithm introduced by [2] with prioritized experience replay, in which the jth past transition from the replay buffer is sampled according to:

$$p^\delta(j) = \frac{(m_j^\delta + \epsilon)^\beta}{\sum_{i=1}^{|D|}(m_i^\delta + \epsilon)^\beta} \tag{2}$$

where $m_j^\delta = |\delta_j|$ is most recent temporal difference error associated with j, and ϵ ensures that each transition always has a positive probability of being sampled and β controls the entropy.

We experiment with augmenting TD priority with measurements of consequence. Given counterfactual consequence estimates from algorithm 1, in analogue with 2, we could compute:

$$p^C(j) = \frac{(m_j^C + \epsilon)^\beta}{\sum_{i=1}^{|D|}(m_i^C + \epsilon)^\beta} \tag{3}$$

where $m_j^C = m(\{d_G^{(\pi)}(s_j, a)\}_{a \in A})$, and set the overall priority as:

$$p(j) = \frac{\mu p^C(j) + (1-\mu)p^\delta(j)}{\sum_{k=1}^{|D|} \mu p^C(k) + (1-\mu)p^\delta(k)} \tag{4}$$

where μ controls the relative contribution of consequence estimates to sampling probabilities. This balanced approach puts the highest priority on transitions that are both important (high consequence) and poorly modeled (high TD error).

Algorithm 2 DQN with balanced consequence-error priority

Input: $\mu, \gamma, \alpha, \epsilon, \beta, M, B_{est}^C, B_{up}, K_{up}, K_{tar}$
Init: Q network parameters $\mathbf{w}$, replay buffer $D = \emptyset$
Set $\mathbf{w}' = \mathbf{w}$ and $\pi(s)$
Sample $s_0 \in S$
for $t = 1, 2, 3, \ldots$ **do**
  Sample a_t according to $\pi(s_t)$, observe r_{t+1} and s_{t+1}
  Add $(s_t, a_t, r_{t+1}, s_{t+1})$ to D
  Set $m_t^C = \sum_{j=1}^{|D|} m_j / |D|$
  Set $m_t^\delta = \max_{j=1,\ldots,|D|}\{m_j\}$
  if $|D| > M$ **then** drop the oldest
  if $t \mod K_{up} = 0$ **then**
    Sample $(s_j, a_j, r_j, s_j')_{j=1}^{B_{est}^C}$ from $U(D)$
    Update m_j^C via algorithm 1 and then update $p(j)$
    Sample $(s_j, a_j, r_j, s_j')_{j=1}^{B_{up}}$ from D via $p(j)$
    Compute importance sampling weights so that:

$$w_j = (p(j)|D|)^{-1}$$

    Compute TD error:

$$\delta_j \leftarrow r_j + \gamma \max_a \left\{Q_\pi(s_j', a|\mathbf{w}')\right\} - Q(s_j, a_j|\mathbf{w})$$

    Update TD priorities: $m_j^\delta \leftarrow |\delta_j|$
    Update Q network weights:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \left(\frac{1}{B_L} \sum_{j=1}^{B_L} w_j \delta_j^2 \right)$$

  end if
  if $t \mod K_{tar} = 0$ **then** $\mathbf{w}' \leftarrow \mathbf{w}$
end for

VI. Experiments

Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar.

[1] There are many options to compare distributions. For example, if we used KL-divergence we could set:

$$m(a_t, a) = \int_{-\infty}^{\infty} d_G^{(\pi)}(g|s_t, a_t) \ln \left\{ \frac{d_G^{(\pi)}(g|s_t, a_t)}{d_G^{(\pi)}(g|s_t, a)} \right\} dg.$$

We consider multiple metrics in our experiments below.

TABLE I
HYPERPARAMETERS

Hyperparameter	***Space***	***Description***
μ	$[0, 1]$	Weight on consequence metric
γ	$[0, 1)$	Discount factor
α	$(0, 1]$	Step size
ϵ	$\mathbb{R}_{\geq 0}$	Priority shaping parameter
β	$\mathbb{R}_{\geq 0}$	Priority shaping parameter
B_{est}^{C}	$\mathbb{N}_1$	Batch size for consequence estimates
B_{up}	$\mathbb{N}_1$	Batch size for Q network update
M	$\mathbb{N}_1$	Replay buffer memory size
K_{up}	$\mathbb{N}_1$	Frequency of Q network updates
K_{tar}	$\mathbb{N}_1$	Fequency of target network updates

VII. Future Work

Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar.

VIII. Conclusion

Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar. Blar blar blar blar.

Acknowledgment

The authors would like to thank the Milwaukee School of Engineering for supporting this research through computational resources and faculty guidance. This work was completed as part of the undergraduate research curriculum in the Department of Computer Science and Software Engineering.

References

[1] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[2] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: https://arxiv.org/abs/1312.5602.

[3] Tom Schaul et al. *Prioritized Experience Replay*. 2016. arXiv: 1511.05952 [cs.LG]. URL: https://arxiv.org/abs/1511.05952.